Control de versions, documentació i integració continua

Introducció i comandos bàsics

Desplegament d'aplicacions Web
2 DAW

Índex

- Sistema de control de Control de Versions
- Tipus de Sistemes de Control de Versions
- Fonaments de GIT
- Comandos bàsics GIT (repositori local)
  - Creació repositori local (git init)
  - Afegir fitxers a l'àrea de preparació (git add)
  - Passar fitxers al repositori local (git commit)
  - Consultar l'estat del directori de treball (git estatus)
  - Consultar diferències entre fitxers (git diff)
  - Consultar l'historial de commits (git log)
  - Evitar "rastrejar" determinats fitxers (arxiu .gitignore)
  - Eliminar arxius del directori de treball (git rm)
  - Desfer canvis (git reset, git revert)

Sistema de control de versions

Què és un SCV?

- Sistema que registra els canvis realitzats sobre un arxiu/s o al llarg del temps de manera que és possible recuperar versions específiques.
- Ha de ser capaç de proporcionar, com a mínim:
  - Mecanisme d'emmagatzematge dels elements que haja de gestionar.
  - Possibilitat de fer canvis sobre els elements emmagatzemats.
  - Registre històric de les accions realitzades amb cada element o conjunt d'elements.

Sistema de control de versions

Per què ho necessitem?

- Facilita tasques de marxa arrere o correcció d'errors.
- Facilita la compartició de codi.
- Accés al registre de modificacions sobre un fitxer (persona responsable i possibles comentaris).
- Creació de branques per a gestionar canvis que finalment es mesclaran amb la branca principal



Sistema de control de versions

SCV locals

- Consisteix a copiar els arxius a un altre directori (indicant, per exemple, dates i hora de creació per a poder mantindre un historial)
- És molt comú perquè és molt **senzill**, **però** també és **tremendament propens a errors**.
- Versió millorada -> passar a emmagatzemar les diferents versions en una base de dades juntament amb els respectius canvis de cada versió (RCS en MacOS).

Sistema de control de versions

SCV centralitzats

- Apareixen per a solucionar el gran problema amb el qual es trobaven les persones que necessitaven col·laborar amb desenvolupadors en altres sistemes.
- Tenen un **únic servidor que conté tots els arxius versionats** i diversos clients que descarreguen els arxius des d'eixe lloc central.
- Ha sigut l'estàndard per al control de versions durant molts anys.

Sistema de control de versions

SCV centralitzats

- **Avantatges enfront de SCV locals:**
  - Totes les persones saben fins a un cert punt en què estan treballant els altres col·laboradors del projecte.
  - Més fàcil d'administrar que haver de bregar amb bases de dades locals en cada client
- **Desavantatges**:
  - Punt únic de fallada que representa el servidor centralitzat. Quan tens tota la història del projecte en un mateix lloc, t'arrisques a perdre'l tot (cosa que també ocorre en els SCV locals).

Sistema de control de versions

SCV centralitzats (exemples)

- CVS (Current Versions System).
  - Desenvolupat en els anys 80, va ser el **primer sistema de control de versions de codi obert (llicència GNU).**
  - Té una arquitectura client-servidor, en la qual el codi està emmagatzemat en un servidor central i amb el programari client podem fer una còpia del codi local per a fer canvis i posteriorment tornar a pujar-la al servidor.
  - El servidor només accepta actualitzacions dels fitxers que estiguen en l'última versió. Això implica que els usuaris han d'actualitzar les seues còpies locals abans de pujar-les al servidor.

Sistema de control de versions



SCV centralitzats (exemples)

- SVN. El projecte de Subversion va sorgir l'any 2000 amb l'objectiu crear un sistema de control de versions amb la **mateixa filosofia que CVS, però arreglant problemes i cobrint mancances** d'aquest, com per exemple:
  - Commits atòmics, en CVS un commit interromput pot deixar dades inconsistents.
  - La creació de branques és més eficient, amb complexitat constant a diferència de CVS que és lineal (augmenta amb el nombre de branques).
  - Maneig d'arxius binaris com a tal, CVS els tracta com a arxius de text.
  - Envia els increments dels fitxers en la comunicació client - servidor, en lloc dels fitxers complets com CVS.

Sistema de control de versions

SCV distribuïts

- Ofereixen solucions per als problemes que han sigut esmentats
- No hi ha un repositori central. Tots els desenvolupadors tenen la seua pròpia còpia del repositori, amb totes les versions i tota la història.
- Segons van desenvolupant i fent canvis, les seues fonts i versions van sent distintes les unes de les altres. No obstant això, permeten que en qualsevol moment dos desenvolupadors qualssevol puguen "sincronitzar" els seus repositoris. Si un dels desenvolupadors ha tocat determinades fonts i l'altre no, els modificats es converteixen en la versió més moderna.
- Si un servidor deixa de funcionar qualsevol dels repositoris disponibles en els clients pot ser copiat al servidor amb la finalitat de restaurar-lo. Cada clon és realment una còpia completa de totes les dades.

Sistema de control de versions

SCV distribuïts (ejemplos)

- Mercurial
- Bazaar
- BitKeeper
- Git





https://www.mercurial-scm.org/



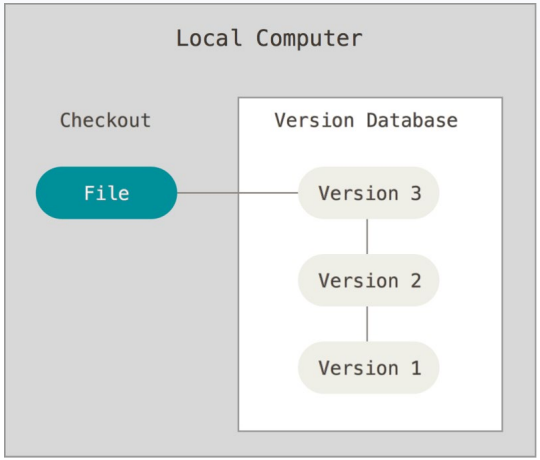
http://bazaar.canonical.com/en/



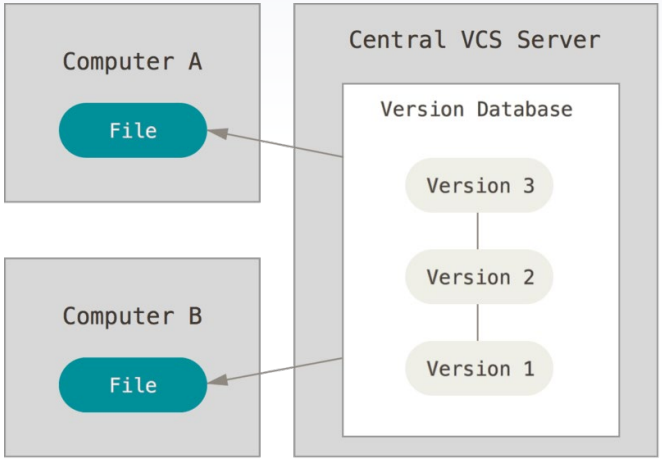
https://www.bitkeeper.org/

Tipus de Sistemes de Control de Versions

Local

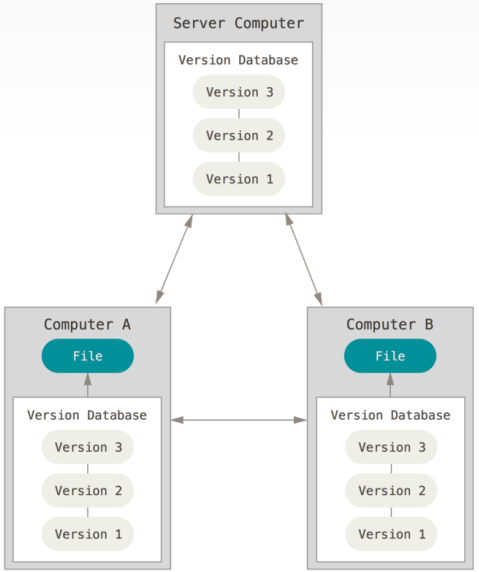


Centralitzat





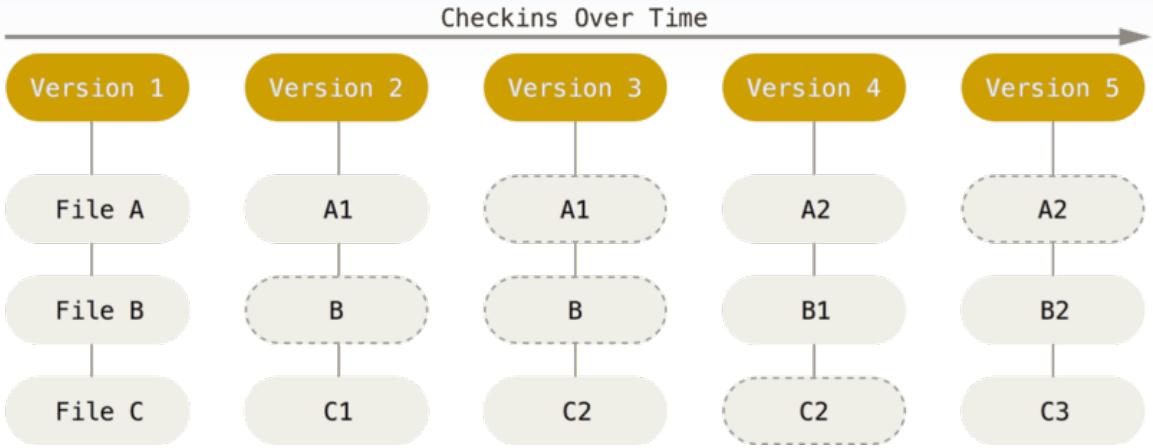
Distribuït





Fonaments de GIT

- **Instantànies, no copies completes**

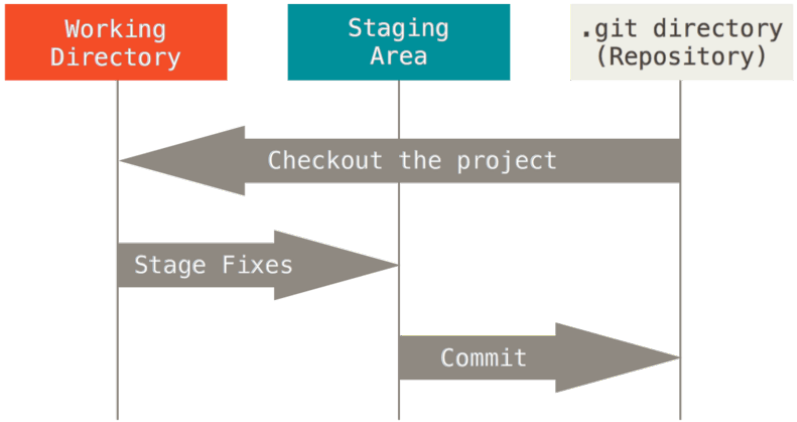


Fonaments de GIT

- **Instantànies**
  - Inclou l'estat exacte dels arxius en un moment donat.
  - Guarda només els arxius modificats, els arxius no modificats es referencien des de versions anteriors.
  - És la base de cada commit, permetent a Git restaurar el teu projecte a l'estat exacte d'eixe commit en qualsevol moment.

Fonaments de GIT

- **Majoria d'operacions locals**
- **Integritat**
  Git no emmagatzema informació per nom d'arxiu, sinó pel valor hash dels seus continguts.
- **3 Estats, 3 àrees**
  - Modificat (modified)
  - Preparat (staged)
  - Confirmat (Committed)



Comandos bàsics de GIT

Creació d'un repositorio local

```
git init
```

Executat a l'interior d'un directori, crea la carpeta .git en la qual s'emmagatzema el contingut necessari perquè Git "sàpia" que es tracta d'un repositori.

```
root@antoni-virtualbox:/home/antoni/Escriptori# mkdir ProjecteGit
root@antoni-virtualbox:/home/antoni/Escriptori# cd ProjecteGit/
root@antoni-virtualbox:/home/antoni/Escriptori/ProjecteGit# sudo git init
consell: S'està utilitzant «master» com a nom de la branca inicial. Aquest nom de branca
consell: per defecte es pot canviar. Per a configurar el nom inicial de la branca que
consell: s'utilitzarà en tots els repositoris nous, i que suprimirà aquest avís, useu:
consell:
consell:        git config --global init.defaultBranch <nom>
consell:
consell: Els noms més usats habitualment en lloc de «master» són «main», «trunk» i
consell: «development». La branca acabada de crear es pot canviar de nom amb l'ordre:
consell:
consell:        git branch -m <nom>
S'ha inicialitzat un repositori buit del Git en /home/antoni/Escriptori/ProjecteGit/.git/
root@antoni-virtualbox:/home/antoni/Escriptori/ProjecteGit#
```

També podem utilitzar git init amb l'opció --bare per a crear un repositori no destinat a treball regular. Simplement com a servidor.

Comandos bàsics de GIT

Afegir fitxers a l'àrea de preparació

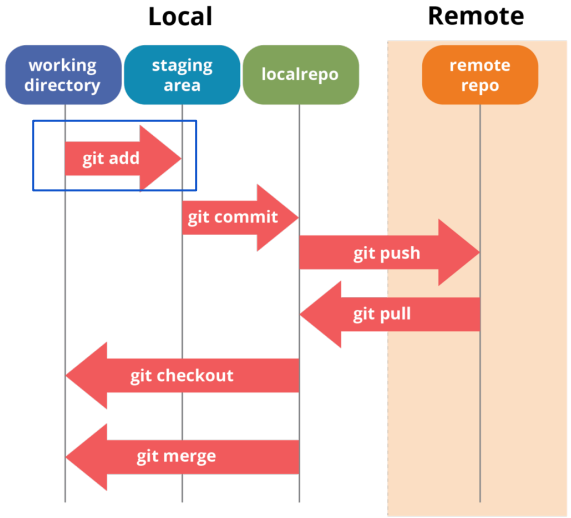
`git add`

Afig fitxers locals a la nostra àrea de preparació. Això és, els passa a estat preparat

git add README.md

git add .

Admet tant el pas del nom d'un fitxer concret com un patró



Algunes opcions interessants:

-u només afig fitxers que ja estan sent monitorats

-n simula que faria git add sense actualitzar res.

Comandos bàsics de GIT

Afegir fitxers al repositori local

`git commit`

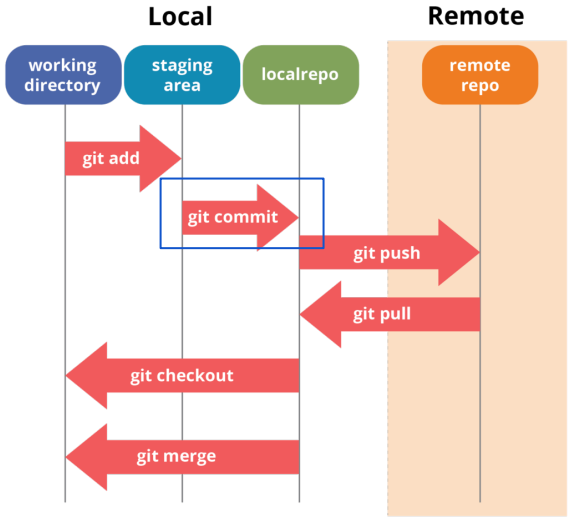
Confirma arxius en estat preparat

git commit (S'obrirà el nostre editor per defecte per a introduir un missatge de confirmació)

git commit -m “Missatge de confirmació” .

Algunes opcions interessants:

- **-- amend** permet afegir nous arxius a l'últim commit realitzat. (--no-edit si no volem modificar el missatge de commit)
- **-am** = git add + git commit en un solo paso

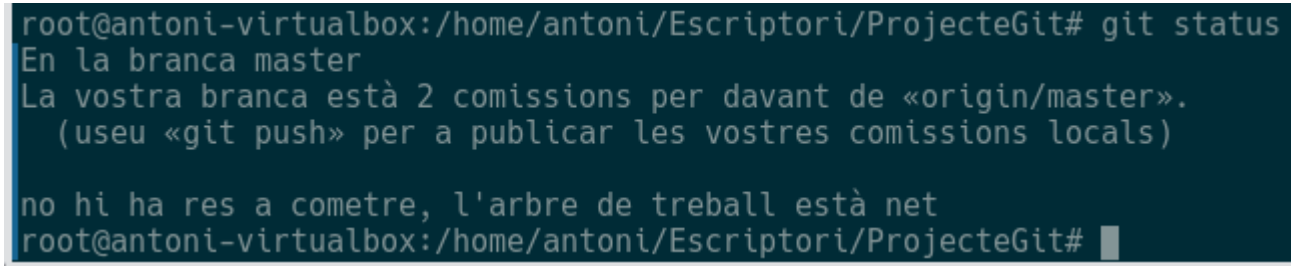


Comandos bàsics de GIT

Consultar l'estat del directori de treball (abans de pujar-ho al servidor)

```
git status
```

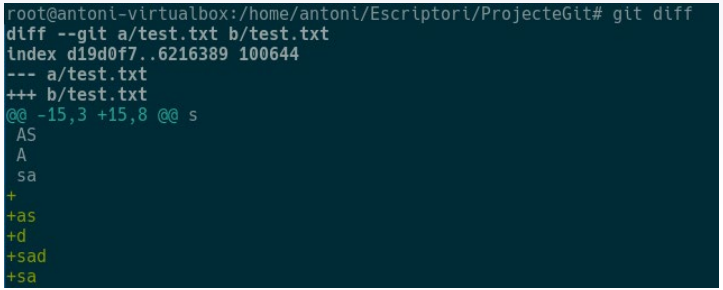
Permet veure els canvis que s'han preparat, els que no i els arxius en els quals Git no realitzarà el seguiment



Comandos bàsics de GIT

Revisar les diferències entre diferents versions dels nostres arxius (abans de fer git add .)

`git diff`



Algunas opciones interesantes:

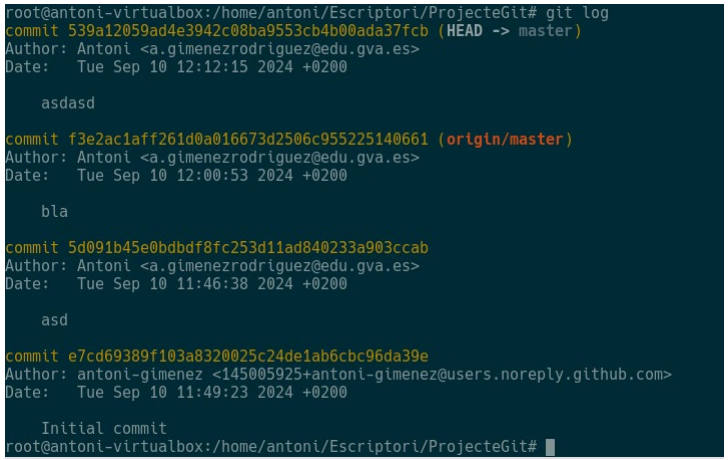
- **git diff (sense paràmetres)**: Revisa les diferencia entre el nostre entorn de treball i la nostra àrea de preparació
- **git diff --staged o --cached**: revisa les diferències entre la nostra àrea de preparació i l'última confirmació o commit
- **git diff nombreRama1 nombreRama2**: revisa diferències entre dues branques distintes.

Comandos bàsics GIT

Consultar l'historial de commits

```
git log
```

El comando git log ens permet llistar l'historial de commits realitzats en el nostre repositori local



D'un commit podem veure diverses informacions bàsiques com:

- Identificador del commit
- Autor
- Data de realització
- Missatge enviat

https://desarrolloweb.com/articulos/git-log.html

Comandos bàsics GIT

Arxiu per a evitar “rastrejar” determinats fitxers/directoris

`.gitignore`

Arxiu que llista patrons que no volem rastrejar

Quins arxius i carpetes hauries de col·locar ací?:

- Arxius que tinguen credencials o claus de API (no hauries de pujar-les al repositori, simplement injectar-les per variable d'entorn)
- Carpetes de configuració del teu editor (/.vscode)
- Arxius de registre (log files)
- Arxius de sistema com .DS_Store
- Carpetes generades amb arxius estàtics o compilacions com /dist o /build
- Dependències que poden ser descarregades (/node_modules)
- Coverage del testing (/coverage)

Comandos bàsics GIT

Arxiu per a evitar “rastrejar” determinats fitxers/directoris

`.gitignore`

```
# ignora los archivos terminados en .a
*.a

# pero no lib.a, aun cuando había ignorado los archivos terminados en .a en la línea anterior
!lib.a

# ignora unicamente el archivo TODO de la raiz, no subdir/TODO
/TODO

# ignora todos los archivos del directorio build/
build/

# ignora doc/notes.txt, pero no este: doc/server/arch.txt
doc/*.txt

# ignora todos los archivos .txt del directorio doc/
doc/**/*.txt
```

GitHub manté una extensa llista d'arxius .gitignore adequats a dotzenes de projectes i llenguatges en https://github.com/github/gitignore

Fins i tot existeix un cercador de fitxers

.gitignore (link)

Comandos bàsics de GIT

Eliminar arxius del directori de treball

```
git rm
```

S'encarrega d'eliminar un arxiu del nostre directori de treball, i a més l'afig a l'àrea de preparació.

També resulta molt útil quan volem canviar de nom un arxiu que ja estem rastrejant en Git.

Algunes opcions interessants:

- **—- cached** elimina l'arxiu de l'àrea de preparació (deixa de rastrejar-lo) però el deixa en el directori de treball
- **-- f** elimina l'arxiu de l'àrea de preparació i de l'àrea de treball

Comandos bàsics de GIT

Desfer canvis

```
git reset
```

Permet desfer canvis que estiguen tant en l'àrea de preparació com en la d'arxius confirmats (de l'últim commit).
Mou tant les referències de HEAD com les de la branca a la confirmació especificada.

git reset nomArxiu -> elimina de l'àrea de preparació nomArxiu (deixa de rastrejar-ho)

git reset --hard HEAD~1 -> elimina l'últim commit i els seus canvis es perdran per complet

git reset --soft HEAD~1 -> elimina l'últim commit però tots els canvis es mantenen en àrea de preparació.

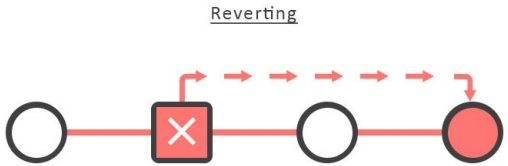
Comandos básicos de GIT

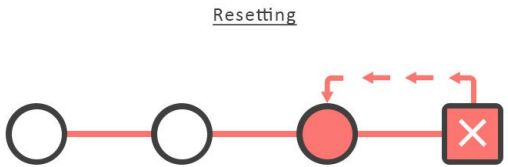
Desfer canvis

```
git revert
```

En lloc d'eliminar el commit de l'historial del projecte, resol com invertir els canvis introduïts per dit commit i afig un nou amb el contingut invers resultant (no movem la posició del HEAD).

Es considera una opció més segura que el git reset







https://www.atlassian.com/es/git/tutorials/undoing-changes/git-revert